

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 孙广润, 刘钰, 蔡合森 箱子编号 18 实验项目编号 实践任务 14,15,16
实验名称 AXI 总线接口设计

一、 类 SRAM 总线

• 代码修改说明。

对于读操作来说, 主方提出读请求, 从方接收请求并返回数据; 对于写操作来说, 主方提出写请求并发出数据, 从方接收请求和数据。我们首先要定义握手信号, 方便上述的后续过程。

```
1 //添加握手信号
2 wire IF_valid;
3 wire IF_allowin;
4 wire IF_readygo;
```

第一次握手逻辑如下, 特别需要强调的是我们的命名在实验 15/16 發生更改, 将 __woshou 的信号进行修改为 __rcv。在后文不再赘述。

```
1 reg inst_first_woshou;
2 always @(posedge clk) begin
3     if (reset) begin
4         inst_first_woshou <=1'b0;
5     end
6     else if(IF_readygo) begin
7         inst_first_woshou <=1'b0;
8     end
9     else if(inst_sram_addr_ok&&inst_sram_req) begin
10        inst_first_woshou <=1'b1;
11    end
12 end
13 reg data_first_woshou;
14 always @(posedge clk) begin
15     if (reset) begin
16         data_first_woshou <=1'b0;
17     end
18     else if(MEM_readygo) begin
19         data_first_woshou <=1'b0;
20     end
21     else if(data_sram_addr_ok&&data_sram_req) begin
22         data_first_woshou <=1'b1;
23     end
24 +end
```

考虑到跳转逻辑信号不会保存, if 级不会立即跳转, 我们对于跳转信号进行储存来等待 if 级。否则后续信号会重置, 影响后续的流程。

```
1 reg br_taken_ID_r;
2 reg br_taken_EX_r;
3 reg [31:0]nextpc_r;
4 always @(posedge clk) begin
5     if (reset) begin
6         br_taken_ID_r <=1'b0;
7         nextpc_r <=32'h0;
8         br_taken_EX_r <=1'b0;
9     end
10    else if(exc_at_EX)begin
11        nextpc_r <=br_target;
12        br_taken_EX_r <=br_taken_EX;
```

```

13     end
14     else if(has_int||exc_at_ID)begin
15         br_taken_ID_r <=br_taken_ID;
16         nextpc_r <=br_target;
17     end
18     else if(IF_readygo)begin
19         br_taken_ID_r <=1'b0;
20         nextpc_r <=32'h0;
21         br_taken_EX_r <=1'b0;
22     end
23     else if(ID_readygo &&!IF_readygo &&ID_valid) begin
24         br_taken_ID_r <=br_taken_ID;
25         nextpc_r <=br_target;
26         br_taken_EX_r <=br_taken_EX;
27     end
28 end

```

下面是对于指令的储存,原因同上。

```

1 //指令寄存器
2 reg [31:0]inst_ID;
3 reg is_inst_ID;
4 always @(posedge clk) begin
5     if (reset) begin
6         inst_ID <=32'h0;
7         is_inst_ID <=1'b0;
8     end
9     else if(ID_readygo &&EX_allowin)begin
10         inst_ID <=inst_sram_rdata;
11         is_inst_ID <=1'b0;
12     end
13     else if(IF_readygo &&!ID_allowin)begin
14         inst_ID <=inst_sram_rdata;
15         is_inst_ID <=1'b1;
16     end
17
18 end

```

我们对于请求的发送不会让其一直发送,当握手之后,就不会继续发送,用 inst_first_woshou 信号将其屏蔽;//

```

1 assign inst_sram_req =!inst_first_woshou&&ID_allowin &&!exc_ade; //取值地址异常时不进行取指
2 assign inst_sram_wstrb =4'b0;
3 assign inst_sram_wr = 1'b0;
4 assign inst_sram_size =2'b10;
5 assign inst_sram_addr =pc;
6 assign inst_sram_wdata =32'b0;
7 assign inst =(is_inst_ID)?inst_ID:inst_sram_rdata;

```

值得注意的是,在下一个实验中,我们将接口里的内存写使能删掉了,这是多余的,我们的转接桥可以根据传过来的地址自己判断写使能,在这里我们将转换桥如何计算的 wstrb 代码展示如下:

```

1 assign wstrb ={sram_size[1], sram_size[1], (sram_size !=0), 1'b1} <<sram_addr[1:0];

```

对 ID 进行修改,添加了 wr 与 size 信号,wstrb 与 req 信号是由 we 与 en 信号改名而来

```

1 assign data_sram_req_ID =inst_ld_b |inst_ld_h |inst_ld_bu |inst_ld_hu |inst_ld_w |inst_st_b |inst_st_h |inst_st_w;
2 assign data_sram_wr_ID =inst_st_b |inst_st_h |inst_st_w;
3 assign data_sram_size_ID =(inst_ld_b |inst_ld_bu |inst_st_b )?2'b00:(inst_ld_h |inst_ld_hu |inst_st_h)?2'b01:2'b10;
4 assign data_sram_wstrb_ID ={4{inst_st_w}} |{2'b00, {2{inst_st_h}}} |{3'b000, inst_st_b};

```

对于访存同理,添加 wr 与 size 信号,改名 wstrb 与 req 信号,这里直接引用 patch 文件

```

1 always @(posedge clk) begin //访存控制
2     if(reset) begin

```

```

3 - data_sram_en_EX <=1'b0;
4 - data_sram_we_EX <=4'b0;
5 + data_sram_req_EX <=1'b0;
6 + data_sram_wstrb_EX <=4'b0;
7 data_sram_wdata_EX <=32'h0;
8 data_sram_type_tag_EX <=4'b0;
9 + data_sram_size_EX <=2'b0;
10 + data_sram_wr_EX <=1'b0;
11 end
12 else if(ID_valid &&EX_allowin &&ID_readygo) begin
13 - data_sram_en_EX <=data_sram_en_ID;
14 - data_sram_we_EX <=data_sram_we_ID;
15 + data_sram_req_EX <=data_sram_req_ID;
16 + data_sram_wstrb_EX <=data_sram_wstrb_ID;
17 data_sram_wdata_EX <=data_sram_wdata_ID;
18 data_sram_type_tag_EX <=data_sram_type_tag;
19 + data_sram_size_EX <=data_sram_size_ID;
20 + data_sram_wr_EX <=data_sram_wr_ID;
21 end
22 end

```

后续控制信号从 EX 传递下去也是同理。直接引用 patch 文件如下：

```

1 //将一些后续控制信号从EX传递下去
2 always @(posedge clk) begin//访存控制
3     if(reset) begin
4         - data_sram_en_MEM <=1'b0;
5         - data_sram_we_MEM <=4'b0;
6         + data_sram_req_MEM <=1'b0;
7         + data_sram_wstrb_MEM <=4'b0;
8         data_sram_addr_MEM <=32'h0;
9         data_sram_wdata_MEM <=32'h0;
10        data_sram_type_tag_MEM <=4'b0;
11        + data_sram_size_MEM <=2'b0;
12        + data_sram_wr_MEM <=1'b0;
13    end
14    else if(MEM_allowin &&EX_valid &&EX_readygo) begin
15        - data_sram_en_MEM <=data_sram_en_EX;
16        - data_sram_we_MEM <=(data_sram_we_EX <<alu_result[1:0]);
17        + data_sram_req_MEM <=data_sram_req_EX;
18        + data_sram_wstrb_MEM <=(data_sram_wstrb_EX <<alu_result[1:0]);
19        data_sram_addr_MEM <=data_sram_addr_EX;
20        data_sram_wdata_MEM <=data_sram_wdata_EX;
21        data_sram_type_tag_MEM <=data_sram_type_tag_EX;
22        + data_sram_size_MEM <=data_sram_size_EX;
23        + data_sram_wr_MEM <=data_sram_wr_EX;
24    end
25 end

```

我们增加了对于 IF 级的流水控制信号,并且修改了中断逻辑,代码如下：

```

1 //IF流控信号
2 assign IF_valid =1'b1;
3
4 always @(posedge clk) begin
5     if (reset)
6         ID_valid <=1'b0;
7     else if((br_taken_ID_r ||br_taken_EX_r) &&ID_allowin &&IF_readygo &&IF_valid)
8         ID_valid <=1'b0;
9     else if ((br_taken_ID ||br_taken_EX) &&ID_allowin)//分支跳转则把预取的错误指令取消
10        ID_valid <=1'b0;
11    else if(ID_allowin)
12        ID_valid <=IF_valid &&IF_readygo;
13 end
14 //pc传递中的控制信号也做了修改，这里给出patch文件
15 always @(posedge clk) begin

```

```

16     if (reset) begin
17         pc <=32'h1c000000; //trick: to make nextpc be 0x1c000000 during reset
18     end
19 - else if(ID_allowin)begin
20 + else if(ID_allowin &&IF_readygo)begin
21     +     if(br_taken_ID_r||br_taken_EX_r)
22     +         pc <=br_target_r;
23     +     else
24         pc <=nextpc;
25     end
26 end
27 always @(posedge clk) begin
28     if (reset)
29         pc_ID <=32'h1bfffffc;
30 - else if(ID_allowin)
31 + else if(ID_allowin &&IF_readygo)
32     pc_ID <=pc;
33 end

```

在我们和汪老师讨论之后,我们修改了 has_int 和 wb_pc。

has_int 是由 has_int_from_csr 与 ID_valid 产生的信号,把中断绑定到一条到达 ID 的指令上,以前我们的中断没有与指令进行协同,CSR 寄存器里中断有效位传过来了我们的处理器就直接处理中断了,哪怕甚至当前流水线里没指令,但是这显然是不合理的,很难去追踪触发中断时的 wb_pc,把中断的判断与 ID_valid 绑定,就可以做到大道至简了

```

1 assign has_int =has_int_from_csr &&ID_valid;
2
3 assign wb_pc =(exc_ade) ?pc :
4               (exc_ale) ?pc_EX :
5               pc_ID;

```

二、 类 SRAM-AXI 的转接桥设计

我们对我们的结构进行了一些改进,我们将 CPU 修改成了 mycputop 的一个模块并且命名为 CPU_core,具体代码修改如下:

1> 在 cpu 部分的修改

```

1 -module mycpu_top(
2 +module cpu_core(
3     input wire      clk,
4     input wire      reseta,
5     // inst sram interface
6     output wire     inst_sram_req,
7     output wire     inst_sram_wr,
8 - output wire [1:0] inst_sram_size,
9 - output wire [3:0] inst_sram_wstrb,
10 + output wire [1:0] inst_sram_size,
11     output wire [31:0] inst_sram_addr,
12     output wire [31:0] inst_sram_wdata,
13 + input wire [31:0] inst_sram_rdata,
14     input wire      inst_sram_addr_ok,
15     input wire      inst_sram_data_ok,
16 - input wire [31:0] inst_sram_rdata,
17     // data sram interface
18     output wire     data_sram_req,
19     output wire     data_sram_wr,
20 - output wire [1:0] data_sram_size,
21 - output wire [3:0] data_sram_wstrb,
22 + output wire [1:0] data_sram_size,
23     output wire [31:0] data_sram_addr,
24     output wire [31:0] data_sram_wdata,

```

```

25 + input wire [31:0] data_sram_rdata,
26     input wire      data_sram_addr_ok,
27     input wire      data_sram_data_ok,
28 - input wire [31:0] data_sram_rdata,
29     // trace debug interface
30     output wire [31:0] debug_wb_pc,
31     output wire [3:0] debug_wb_rf_we,

```

2> 在 mycpu_top 的修改

```

1 +cpu_core u_cpu_core (
2 +     .clk(ac1k),
3 +     .resetn(aresetn),
4 +
5 +     .inst_sram_req(inst_req),
6 +     .inst_sram_wr(inst_wr),
7 +     .inst_sram_size(inst_size),
8 +     .inst_sram_addr(inst_addr),
9 +     .inst_sram_wdata(inst_wdata),
10 +    .inst_sram_rdata(inst_rdata),
11 +    .inst_sram_addr_ok(inst_addr_ok),
12 +    .inst_sram_data_ok(inst_data_ok),
13 +
14 +    .data_sram_req(data_req),
15 +    .data_sram_wr(data_wr),
16 +    .data_sram_size(data_size),
17 +    .data_sram_addr(data_addr),
18 +    .data_sram_wdata(data_wdata),
19 +    .data_sram_rdata(data_rdata),
20 +    .data_sram_addr_ok(data_addr_ok),
21 +    .data_sram_data_ok(data_data_ok),
22 +
23 +    .debug_wb_pc(debug_wb_pc),
24 +    .debug_wb_rf_we(debug_wb_rf_we),
25 +    .debug_wb_rf_wnum(debug_wb_rf_wnum),
26 +    .debug_wb_rf_wdata(debug_wb_rf_wdata)
27 +);

```

• 转接桥设计。

类 SRAM-AXI 转接桥要有两个类 SRAM 接口。分别连接类 sram 的指令与数据部分,对外通常有一个 AXI 总线接口,转接桥需要将类 SRAM 的地址映射到 AXI 的地址空间,当类 sram 的指令或者数据发送到转接桥的时候,转接桥需要将其转化为 AXI 格式的数据,满足 AXI 总线的读写请求。

```

1 module lsram2axi_bridge(
2     input wire      clk,
3     input wire      resetn,
4
5     //inst sram-like
6     input wire      inst_req,
7     input wire      inst_wr,
8     input wire [1:0] inst_size,
9     input wire [31:0] inst_addr,
10    input wire [31:0] inst_wdata,
11    output wire [31:0] inst_rdata,
12    output wire      inst_addr_ok,
13    output wire      inst_data_ok,
14
15    //data sram-like
16    input wire      data_req,
17    input wire      data_wr,
18    input wire [1:0] data_size,
19    input wire [31:0] data_addr,
20    input wire [31:0] data_wdata,
21    output wire [31:0] data_rdata,
22    output wire      data_addr_ok,

```

```

23     output wire    data_data_ok,
24
25     //axi
26     //ar
27     output wire [3:0] arid,// 0 for inst; 1 for data
28     output wire [31:0] araddr,
29     output wire [7:0] arlen,// always 0
30     output wire [2:0] arsize,
31     output wire [1:0] arburst,// always 2'b01
32     output wire [1:0] arlock,// always 2'b0
33     output wire [3:0] arcache,// always 4'b0
34     output wire [2:0] arprot,// always 3'b0
35     output wire    arvalid,
36     input wire    aready,
37     //r
38     input wire [3:0] rid,
39     input wire [31:0] rdata,
40     input wire [4:0] rresp,//ignore
41     input wire    rlast,//ignore
42     input wire    rvalid,
43     output wire    rready,
44     //aw
45     output wire [3:0] awid,// always 4'b1
46     output wire [31:0] awaddr,
47     output wire [7:0] awlen,// always 0
48     output wire [2:0] awsize,
49     output wire [1:0] awburst,// always 2'b01
50     output wire [1:0] awlock,// always 2'b0
51     output wire [3:0] awcache,// always 4'b0
52     output wire [2:0] awprot,// always 3'b0
53     output wire    awvalid,
54     input wire    awready,
55     //w
56     output wire [3:0] wid,// always 4'b1
57     output wire [31:0] wdata,
58     output wire [3:0] wstrb,
59     output wire    wlast,// always 1
60     output wire    wvalid,
61     input wire    wready,
62     //b
63     input wire [3:0] bid,// ignore
64     input wire [4:0] bresp,// ignore
65     input wire    bvalid,
66     output wire    bready
67 );
68
69 reg        reset;
70 always @(posedge clk) reset <=~resetn;
71
72
73 reg sram_req;
74 reg sram_req_id;
75 reg sram_wr;
76 reg [1:0] sram_size;
77 reg [31:0] sram_addr;
78 reg [31:0] sram_wdata;
79
80 wire axi_rd_rps;
81 wire axi_wr_rps;
82 wire axi_response;
83
84 always @(posedge clk) begin
85     if(reset) begin
86         sram_req <=1'b0;
87     end
88     else if((inst_req ||data_req) &&!sram_req) begin
89         sram_req <=1'b1;
90     end

```

```

91     else if(axi_response) begin
92         sram_req <=1'b0;
93     end
94 end
95
96 always @(posedge clk) begin
97     if(reset) begin
98         sram_req_id <=1'b0;
99     end
100     else if(!sram_req) begin
101         sram_req_id <=data_req;
102     end
103 end
104
105 always @(posedge clk) begin
106     if(data_req &&data_addr_ok)begin
107         sram_wr <=data_wr;
108         sram_size <=data_size;
109         sram_addr <=data_addr;
110         sram_wdata <=data_wdata;
111     end
112     else if(inst_req &&inst_addr_ok)begin
113         sram_wr <=inst_wr;
114         sram_size <=inst_size;
115         sram_addr <=inst_addr;
116         sram_wdata <=inst_wdata;
117     end
118 end
119
120 reg [31:0] rdata_r;
121 always @(posedge clk) begin
122     if(reset) begin
123         rdata_r <=32'b0;
124     end
125     else if(sram_req &&axi_rd_rps) begin
126         rdata_r <=rdata;
127     end
128 end
129
130 reg inst_data_readygo;
131 always @(posedge clk) begin
132     if(reset) begin
133         inst_data_readygo <=1'b0;
134     end
135     else if(sram_req &&!sram_req_id &&axi_response &&!inst_data_readygo) begin
136         inst_data_readygo <=1'b1;
137     end
138     else if(inst_data_readygo) begin
139         inst_data_readygo <=1'b0;
140     end
141 end
142 reg data_data_readygo;
143 always @(posedge clk) begin
144     if(reset) begin
145         data_data_readygo <=1'b0;
146     end
147     else if(sram_req &&sram_req_id &&axi_response &&!data_data_readygo) begin
148         data_data_readygo <=1'b1;
149     end
150     else if(data_data_readygo) begin
151         data_data_readygo <=1'b0;
152     end
153 end
154
155 //sram-like
156 assign inst_addr_ok =!sram_req &&!data_req;
157 assign inst_data_ok =inst_data_readygo;
158 assign inst_rdata =rdata_r;

```

```

159 assign data_addr_ok =!sram_req;
160 assign data_data_ok =data_data_readygo;
161 assign data_rdata =rdata_r;
162
163
164 reg addr_rcv;
165 reg wdata_rcv;
166 always @(posedge clk) begin
167     if(reset) begin
168         addr_rcv <=1'b0;
169     end
170     else if(arready &&arvalid ||awvalid &&awready) begin
171         addr_rcv <=1'b1;
172     end
173     else if(axi_response) begin
174         addr_rcv <=1'b0;
175     end
176 end
177 always @(posedge clk) begin
178     if(reset) begin
179         wdata_rcv <=1'b0;
180     end
181     else if(wready &&wvalid) begin
182         wdata_rcv <=1'b1;
183     end
184     else if(axi_response) begin
185         wdata_rcv <=1'b0;
186     end
187 end
188
189 assign axi_rd_rps =addr_rcv &&(rvalid &&rready);
190 assign axi_wr_rps =addr_rcv &&wdata_rcv &&(bvalid &&bready);
191 assign axi_response =axi_rd_rps ||axi_wr_rps;
192
193 //axi
194 assign arid ={3'b0, sram_req_id}; // 0 for inst; 1 for data
195 assign araddr =sram_addr;
196 assign arlen =3'b0; //always 0
197 assign arsize =sram_size;
198 assign arbust =2'b01; // always 2'b01
199 assign arlock =2'b0; // always 2'b0
200 assign arcache =4'b0; // always 4'b0
201 assign arprot =3'b0; // always 3'b0
202 assign arvalid =sram_req &&!sram_wr &&!addr_rcv;
203
204 assign rready =1'b1;
205
206 assign awid =4'b1;
207 assign awaddr =sram_addr;
208 assign awlen =3'b0; //always 0
209 assign awsize =sram_size;
210 assign awburst =2'b01; // always 2'b01
211 assign awlock =2'b0; // always 2'b0
212 assign awcache =4'b0; // always 4'b0
213 assign awprot =3'b0; // always 3'b0
214 assign awvalid =sram_req &&sram_wr &&!addr_rcv;
215
216 assign wid =4'b1;
217 assign wdata =sram_wdata;
218 assign wstrb ={sram_size[1], sram_size[1], (sram_size !=0), 1'b1} <<sram_addr[1:0];
219 assign wlast =1'b1; //always 1
220 assign wvalid =sram_req &&sram_wr &&wdata_rcv;
221
222 assign bready =1'b1;
223
224 endmodule

```


我们在顶层接口文件中分别例化了 `cpu_core` 和转接桥,并将其对应接口一一连接。

```
1 module mycpu_top(  
2     input wire      aclk,  
3     input wire      aresetn,  
4     //ar  
5     output wire [3:0] arid,  
6     output wire [31:0] araddr,  
7     output wire [7:0] arlen,  
8     output wire [2:0] arsize,  
9     output wire [1:0] arburst,  
10    output wire [1:0] arlock,  
11    output wire [3:0] arcade,  
12    output wire [2:0] arprot,  
13    output wire      arvalid,  
14    input wire      arready,  
15    //r  
16    input wire [3:0] rid,  
17    input wire [31:0] rdata,  
18    input wire [1:0] rresp,  
19    input wire      rlast,  
20    input wire      rvalid,  
21    output wire      rready,  
22    //aw  
23    output wire [3:0] awid,  
24    output wire [31:0] awaddr,  
25    output wire [7:0] awlen,  
26    output wire [2:0] awsize,  
27    output wire [1:0] awburst,  
28    output wire [1:0] awlock,  
29    output wire [3:0] awcache,  
30    output wire [2:0] awprot,  
31    output wire      awvalid,  
32    input wire      awready,  
33    //w  
34    output wire [3:0] wid,  
35    output wire [31:0] wdata,  
36    output wire [3:0] wstrb,  
37    output wire      wlast,  
38    output wire      wvalid,  
39    input wire      wready,  
40    //b  
41    input wire [3:0] bid,  
42    input wire [1:0] bresp,  
43    input wire      bvalid,  
44    output wire      bready,  
45  
46    //debug interface  
47    output wire [31:0] debug_wb_pc,  
48    output wire [3:0] debug_wb_rf_we,  
49    output wire [4:0] debug_wb_rf_wnum,  
50    output wire [31:0] debug_wb_rf_wdata  
51 );  
52 //sram-like interface  
53 wire      inst_req;  
54 wire      inst_wr;  
55 wire [1:0] inst_size;  
56 wire [31:0] inst_addr;  
57 wire [31:0] inst_wdata;  
58 wire [31:0] inst_rdata;  
59 wire      inst_addr_ok;  
60 wire      inst_data_ok;  
61  
62 wire      data_req;  
63 wire      data_wr;  
64 wire [1:0] data_size;  
65 wire [31:0] data_addr;  
66 wire [31:0] data_wdata;
```

```

67 wire [31:0] data_rdata;
68 wire      data_addr_ok;
69 wire      data_data_ok;
70
71 cpu_core u_cpu_core (
72     .... //cpu的端口在此一一连接
73 );
74
75 lsram2axi_bridge u_lsram2axi_bridge (
76     ....//桥片的端口在这里一一连接
77 );
78
79 endmodule

```

• 优化指令寄存器的数据通路。

```

1  -assign inst =(is_inst_ID)?inst_ID:inst_sram_rdata;
2  +assign inst =(inst_addr_rcv &&inst_sram_data_ok) ?inst_sram_rdata :inst_reg;
3
4  always @(posedge clk) begin
5      if (reset) begin
6          inst_ID <=32'h0;
7          is_inst_ID <=1'b0;
8          inst_reg <=32'h0;
9      end
10     - else if(ID_readygo &&EX_allowin)begin
11         - inst_ID <=inst_sram_rdata;
12         - is_inst_ID <=1'b0;
13     - end
14     - else if(IF_readygo &&!ID_allowin)begin
15         - inst_ID <=inst_sram_rdata;
16         - is_inst_ID <=1'b1;
17     + else if(inst_addr_rcv &&inst_sram_data_ok)begin
18     + inst_reg <=inst_sram_rdata;
19     end
20 end

```

三、 实验分工

• 实践十四,实践十五,实践十六。

- * 刘钰:实践任务十四;
- * 孙广润:实践任务十五;实践任务十六;实验报告;
- * 蔡合森:实践任务十六,实验报告;